



MALWARE – PART I

Prof. Dr. Bernhard Tellenbach

Content

- Definition & History
- Malware Classification
- Malware Communication
 - Communication architecture
 - Communication Channels
 - Resilience / Stealth

Goals

- You know the **most important «types»** of malware like worms, Trojans, ransomware, rootkits, bootkits,...
- You know several techniques **how malware communicates** and **how malware hides** itself from the prying eyes of users and defenders
- You have a basic understanding of **why our defences** against malware are **still quite weak**

Malware - Overview

Definition

- **Malware** is an acronym for **malicious software**
- It comes in **different formats**; executables, shell code, scripts, firmware, ...
- Two definitions:
 - **NIST** [1]: A program that is inserted into a system, usually covertly, with the intent of compromising the confidentiality, integrity, or availability of the victim's data, applications, or operating system or otherwise annoying or disrupting the victim.
 - **OR-MEIR et al.** [2]: Malware is code running on a computerized system whose presence or behavior the system administrators are unaware of; were the system administrators aware of the code and its behavior, they would not permit it to run. Malware compromises the CIA of the system by exploiting existing vulnerabilities in a system or by creating new ones.
- Most definitions include at least one of the following **two main traits**:
 - **maliciousness** or **harmfulness**
 - the ability to perform actions **without the user's consent** or **knowledge**

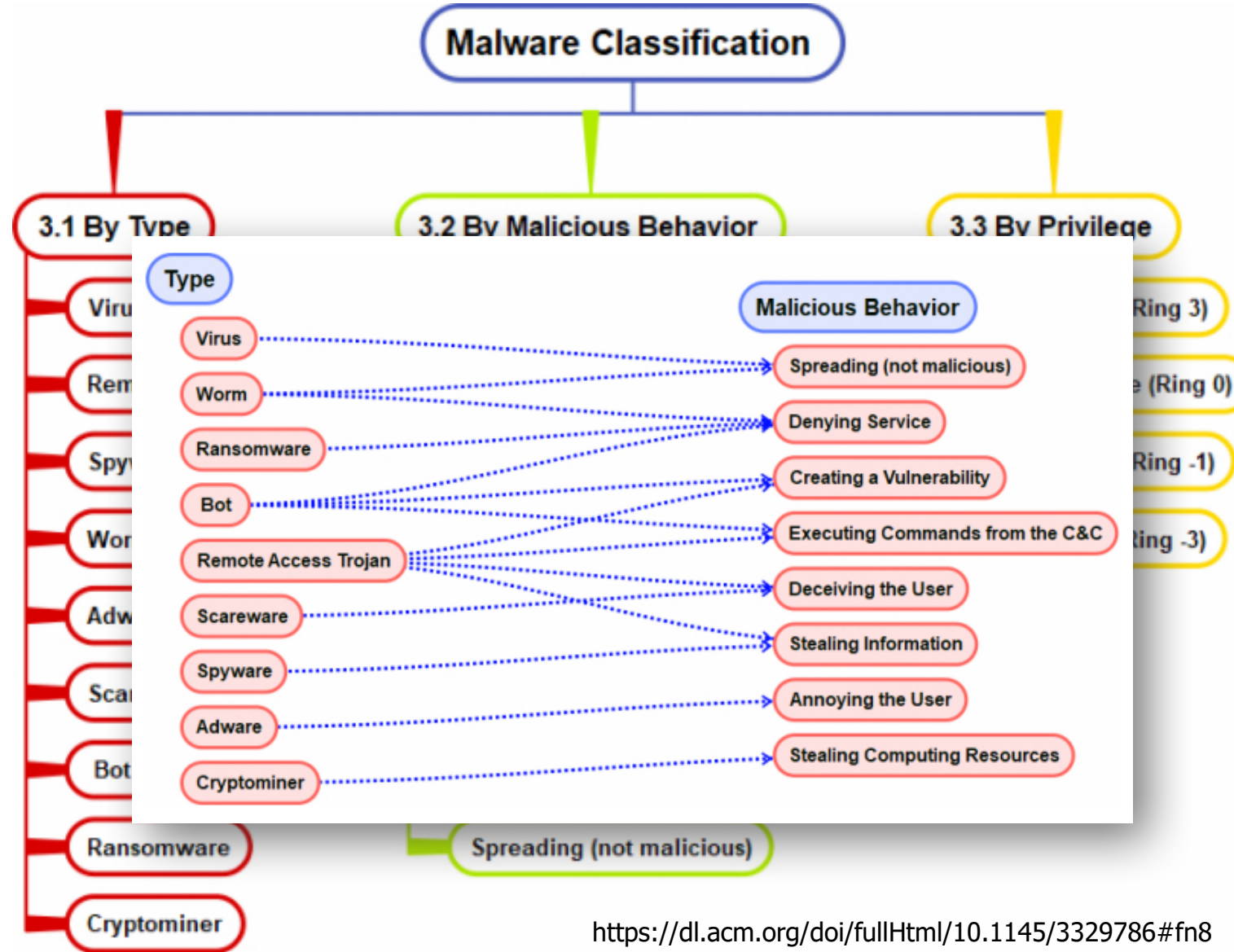
A Brief History of Malware

- 1949 – John von Neumann describes a **design for a self-reproducing computer program** => theoretical father of computer virology
- 1982 – **Elk Cloner**, the **first computer virus** in the wild
 - Written for Apple II by Rich Skrenta, a 15-year-old high school student
 - Boot sector virus spreading to other disks inserted when resident
 - Displayed a poem on every fiftieth boot from an infected disk
- 1988 – **Moris Worm**, the **first computer worm** distributed via the Internet and to gain mainstream media attention
 - Written for Unix by Robert Morris, graduate student at Cornell University
 - Infected **2'000 computers in <15 hours** by exploiting vulnerabilities in sendmail, finger, and rsh/rexec, as well as weak passwords
 - Should do no harm but **multiple infections** of the same machine **slowed them down** to a point of **being unusable**
- 2001 – **Win32.5-0-1**, the **first social networking virus** (MSN Messenger and bulletin boards) written by Matt Larose
 - Required a click to get infected and sent out an email with user data

Malware Classification

- Malware classification helps us in multiple ways, for example to have a **common language** and understanding of a malware and for **risk scoring**
- There are multiple types of classification with different end goals
 - By **type**: Based on the main features implemented by the malware (e.g., replication method)
 - Terms frequently used in mass media, for example bot, virus or worm
 - By **behavior**: Based on the exhibited behavior
 - For example, “information stealer”
 - By **family/lineage**: Focus on the authorship and the lineage / evolution
 - For example, by analyzing the code-base for shared or similar code
- The problem with malware classification: there is **no generic commonly agreed** classification scheme
- For the **types**, there is **some consensus** on typical characteristics of a malware of that type, but details might vary

Malware Classification



Malware Types & Terms

- There is no such thing like mutually exclusive malware types but rather categories of malware that are not mutually exclusive
- Aside from the types below, you should also know what these terms mean: Living Off the Land and Fileless Malware

- | | | |
|----------------------|---------------------|---------------------|
| • Trojan Horse | • Bot/Botnet | • Cryptominer |
| • Backdoor | • Monitor / Spyware | • Spyware |
| • Remote Access Tool | • Mailer Scareware | • Rootkit / Bootkit |
| • Downloader | • Adware | • Worm |
| • Dropper | • Ransomware | • Virus |

Trojan Horse

- Trojan or Trojan Horse is frequently used to classify malware samples that employ some mechanism to conceal their true identity.
- Some examples:
 - A whitepaper you downloaded concealing a PDF exploit
 - A binary version of a 3rd party library containing a backdoor
 - Malware embedded within a bundle that pretends to be a printer driver
 - A music player that contains embedded backdoor code when you run it



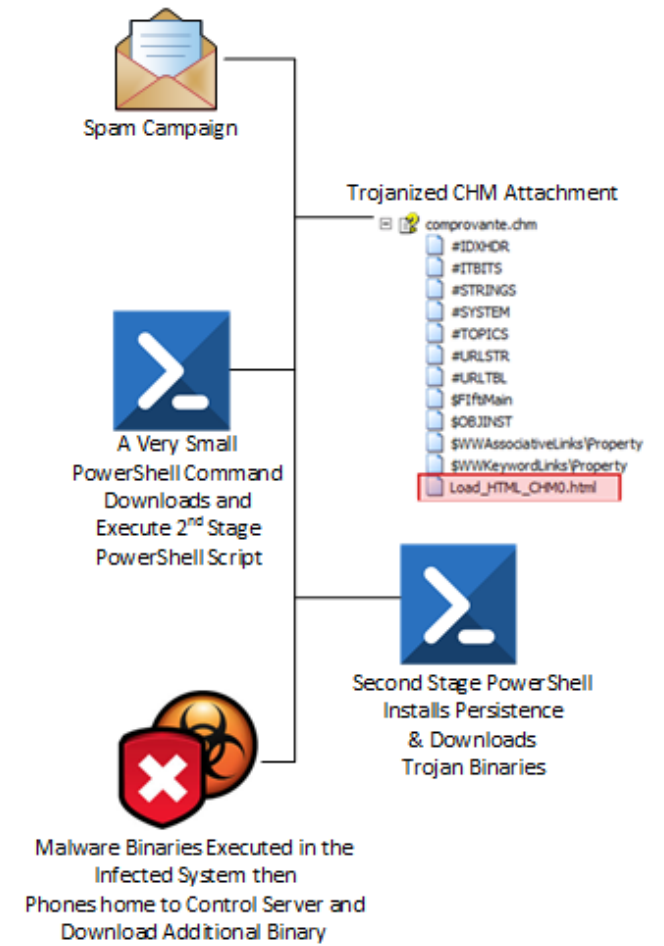
Backdoor & Remote Access Tools (RAT)

- Traditionally, backdoor functionality provides some level of **interactive 1-to-1 access** to a compromised system
 - Usually provided through a **network connection** between adversary and the target's host.
- A Remote Access Tool is frequently used to describe a backdoor having a **high level of interactive functionality** and providing expansive access to the target's host.
 - Sometimes, the level of access provided is **more powerful** than the **normal end-user interface** of the targeted system.



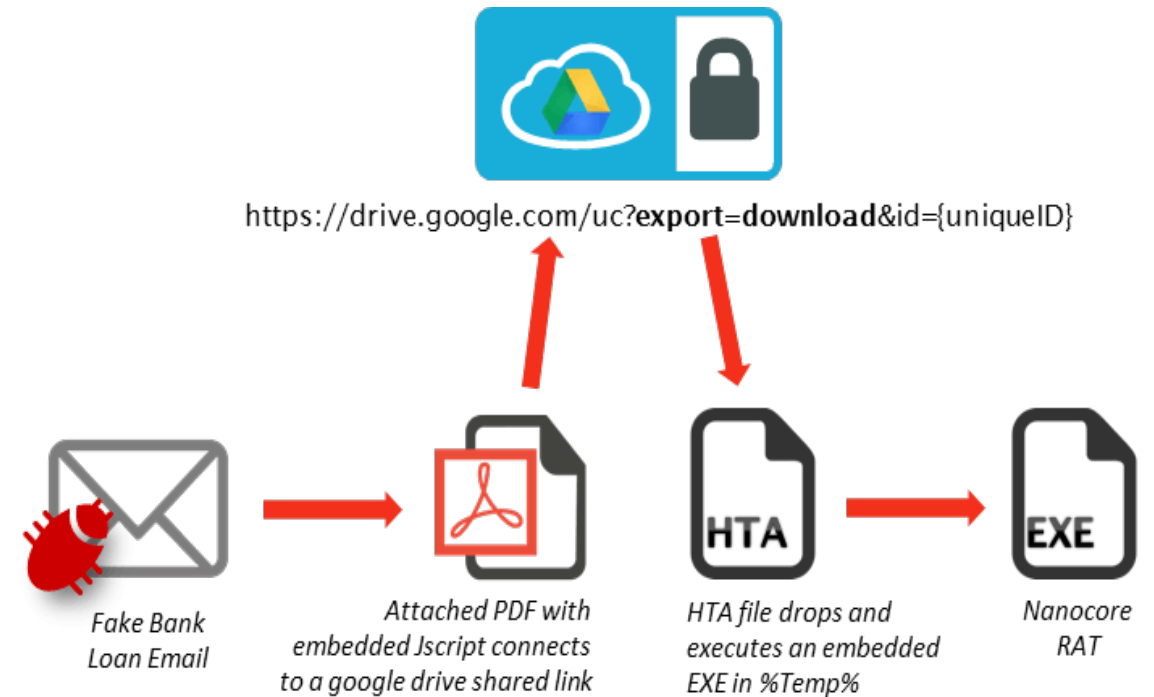
Downloader

- During a mission, attackers frequently employ **multiple tools**
- Downloader functionality provides a tool with a specific feature to **download other tools**
- In the **initial phase** (e.g., spear-phishing), adversaries often deliver **a lightweight tool**
- It may be pre-configured to pull down **more feature-full malware**
 - **Attributes** of the compromised host **may guide** from where and what to retrieve next
- The **breaking-up** of attacks helps to reduce **detectability** and facilitates **surgical retooling**.



Dropper

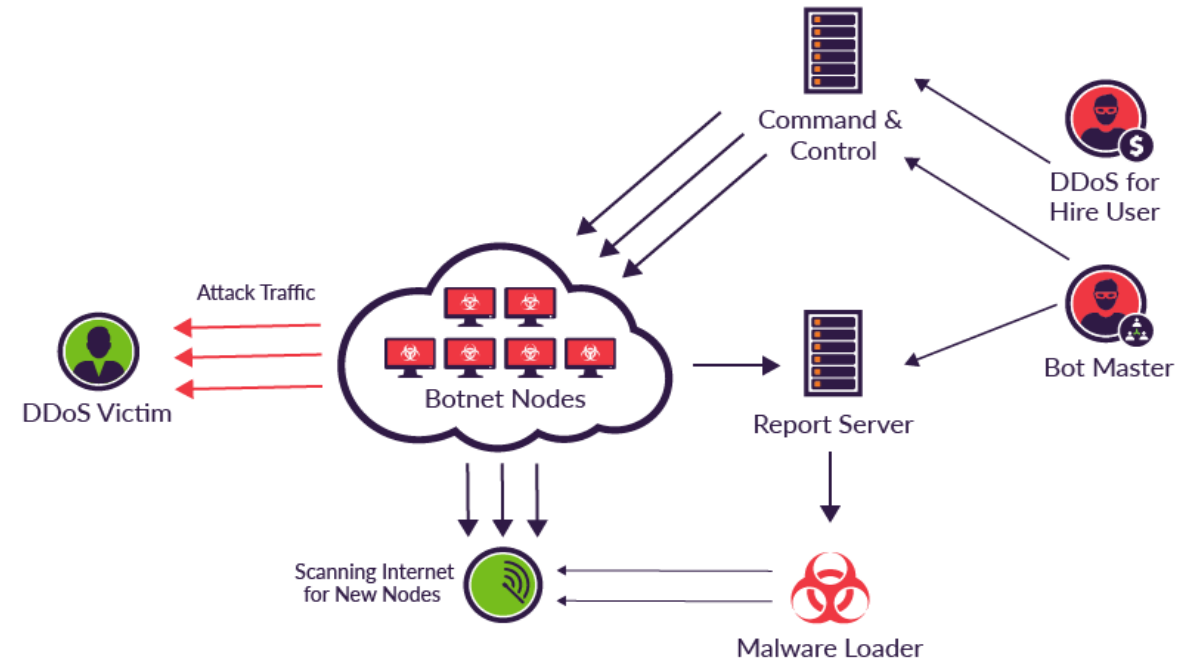
- In a multi-stage attack, one tool may need to **write/drop** another one to disk and execute it.
- Code must first be **extracted and stored** in an OS-compatible, native container (e.g., EXE file).
- Example:
 - HTA document that contains embedded, encoded malware
 - When opened, the embedded malware is extracted, dropped to disk and executed
- **Note:** Downloaders are usually also Droppers, but not vice versa



Bot / Botnets

- **Botnet** - A number of interconnected computers (=nodes) running a **bot**
- The bot is **coordinating** their actions **automatically** or by command and control (**C2/C&C**) from the **"operator"**
- Similar to **Backdoor** but control is usually **1:m** instead of **1:1**
- Common use cases for a botnet and the power of **coordinated actions** are:
 - Distributed Denial of Service (DDoS)
 - Spam campaigns
 - Stealing credentials at scale

Mirai at a Glance



Spyware (Monitor)

- Malware with mechanism to record user activity or the environment employs **monitor** functionality
- The **recorded data** is delivered back to the adversary
- Example monitor sources on a target:
 - Webcam / Microphone
 - Desktop recording
 - Password prompts
 - Common data folders (My Documents, ...)
 - Keyboard (keylogging)
 - Web browser traffic
 - Network traffic

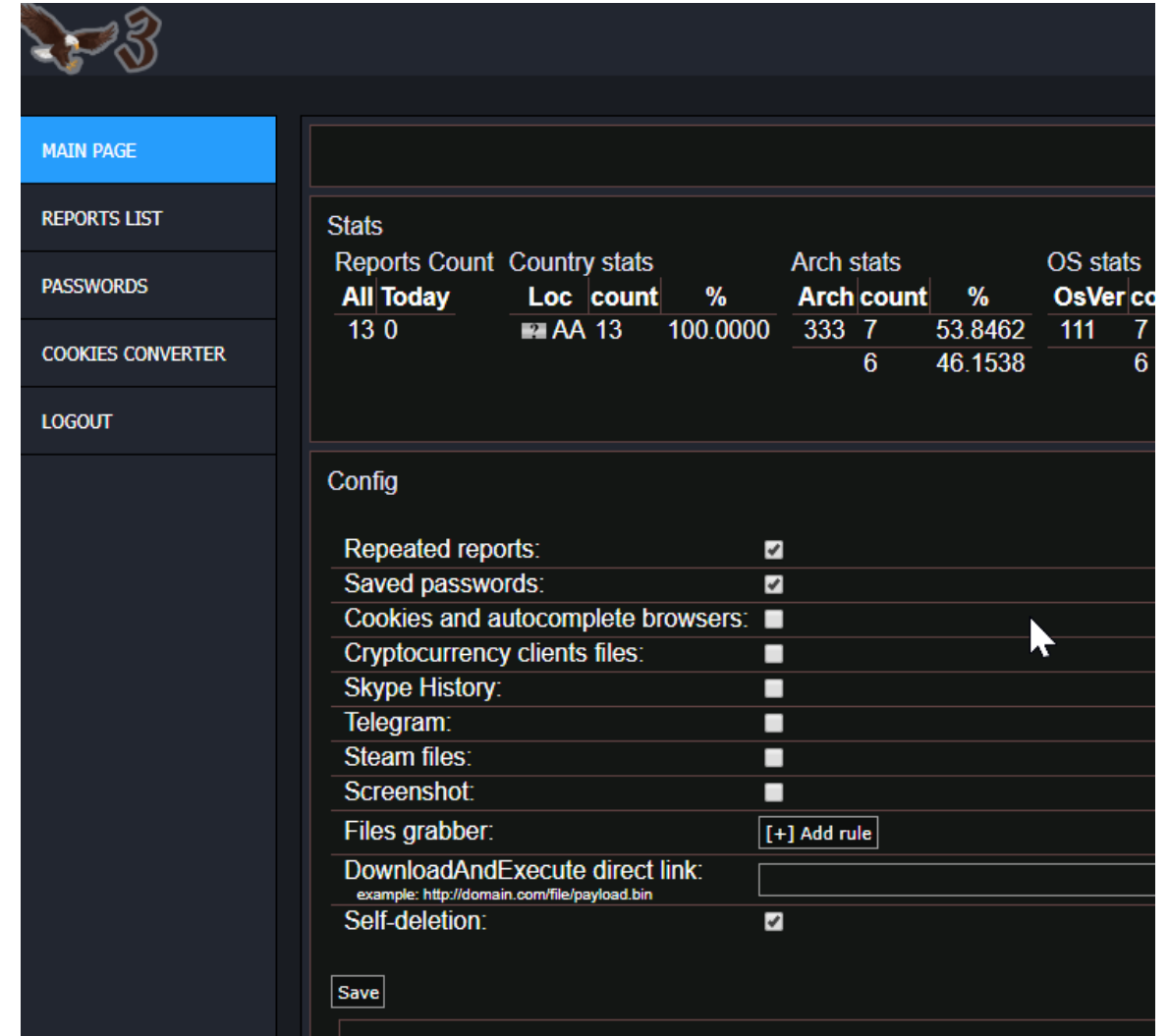
So, What Mobile Phone Spy Features Do I Get with StealthGenie?

With StealthGenie, you have lots of exclusive features for you to monitor any phone remotely and invisibly. You literally have complete access and control of the phone you want to monitor and the best part is, the phone's user will never even know. Below are some of the powerful features. Click on 'View all Features' to view the complete list of StealthGenie features.



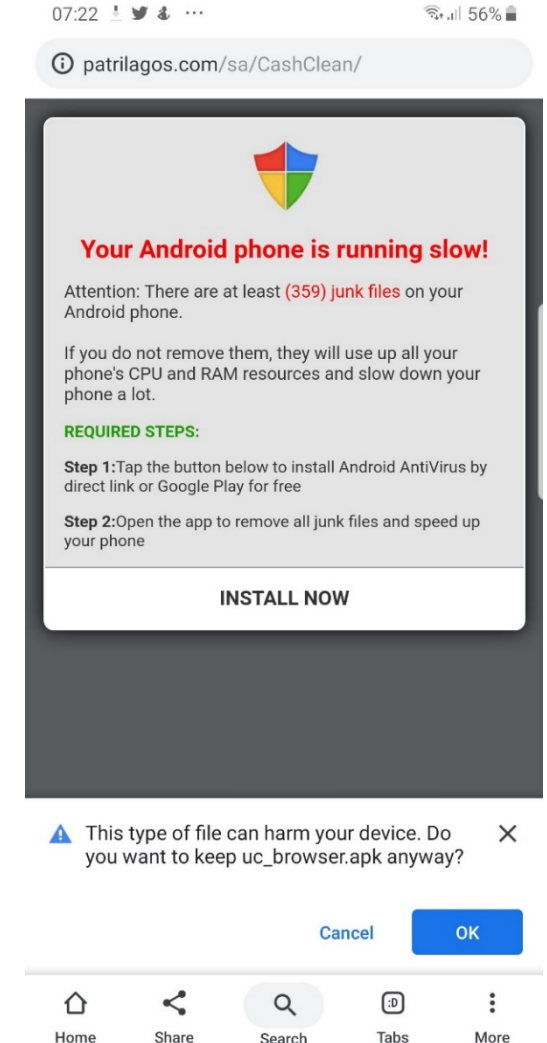
Information Stealer (Spyware)

- Malware with functions to automate the process of stealing information
- It searches data using a hard-coded or network-provided collection plan
- Common approaches include:
 - Contact list theft
 - Browser cookies/history
 - Targeted document theft
 - Credential- / Keystores
- Similar to Monitor but with automated search & extraction of relevant data only



Scareware / Adware

- Employed to achieve a social engineering outcome
- Entices or frequently scares the user into taking specific actions
- A common variant is "Fake Anti Virus" programs
 - When visiting a website, visitors are told they have a virus
 - The site offers a (fake) free Anti-Virus program to remove it
 - It then annoys the user (e.g., with popups) until a fee is paid
- Adware is another common variant where the software may not be an illegal enterprise, but is annoying nonetheless



Ransomware

- Similar end-goals as scareware - **social-engineer** the target into paying a fee or some other activity
- Typically, ransomware **encrypts files** on the local system and **asks for money** to decrypt them
 - Contemporary ransomware leverages strong **public-key cryptography**
 - The malware only contains the public key => makes it practically impossible to recover the files without paying



Virus and Worm

	Virus	Worm
What does it do?	Inserts malicious code into a program or files that can contain «executable code» (e.g., macros, scripts,...)	Exploits a vulnerability in an application, service or operating system*
How does it spread to other computers?	Users transfer infected files to other computers. They are then executed/used there	Uses a network to travel from one computer to another
Does it infect a file?	Yes	No
Does it spread with help of humans?	Yes	No
Example	Sality	WannaCry

- Virus and worm functionality describes approaches to self-propagating/replicating malware (example: WannaCry)
- In practice, distinction might be hard (e.g., email-worm/virus)

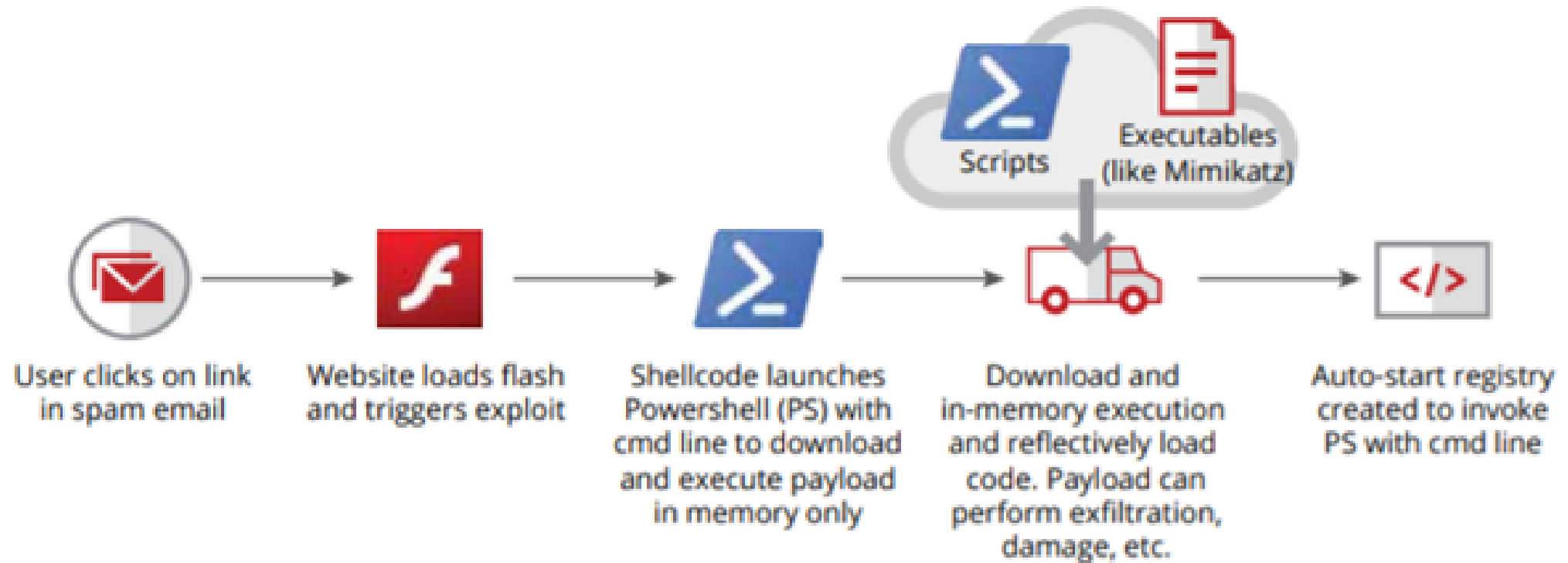
Famous Computer Worms

- **Email Worms** – Malicious attachment or link to infected website
 - ILOVEYOU worm hit 10% of Internet connected computers (50 million infections in 12 days) and caused >15 billions of estimated damage
- **Internet worm** will scan all available network resources using local operating system services and/or scan the Internet for vulnerable machines to connect and gain access to them.
 - Famous examples: Code Red, Blaster, Witty, SQLSlammer, Conficker
 - Witty worm: One UDP packet to rule a security product (ISS firewalls)
- **Fast-spreading worms** (without requiring user interaction)
 - Scanning for infected hosts (often, local network first, then the Internet)

Other Terms and Types in brief:

- **Mailer (or Spambot)** – Malware with functionality to send emails
 - Deliver emails directly via SMTP or using web-based mail accounts of the victim
- **Cryptominer / Cryptojacking** – Malware with functionality to mine crypto currencies. Impact is usage of your resources for free.
- **Living Off the Land** - Malware that utilizes already installed software to implement some/all of its functionality (e.g., PowerShell or a legitimate remote management software)
- **Fileless Malware** – No stand-alone malware is installed
 - Malware code is only in memory, not on disk/in files
 - **In-memory** only: Exploit vulnerability in a software to inject the malware into that process from remote without any file-system access
 - Software that is **already installed** is leveraged to load malware code into memory (subset of living off the land)

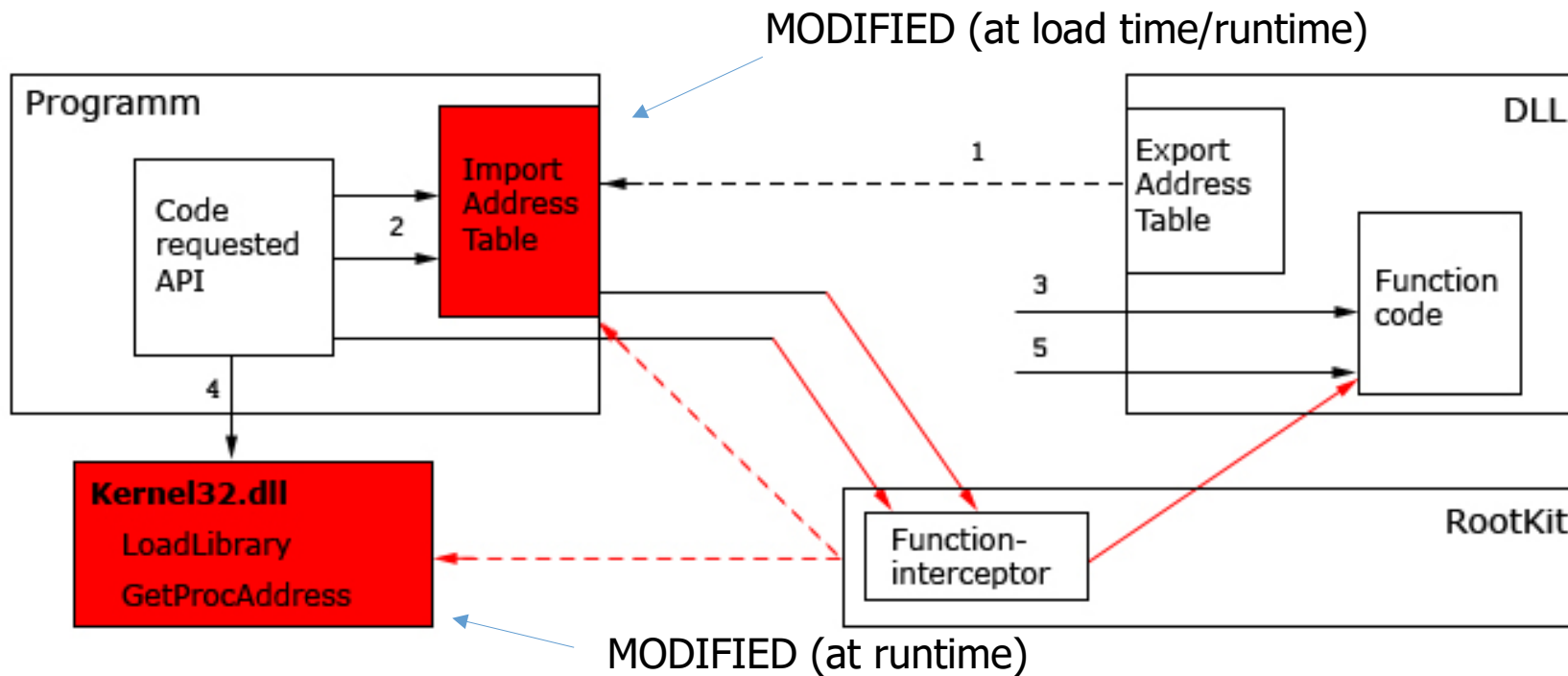
Fileless or not Fileless?



Rootkits

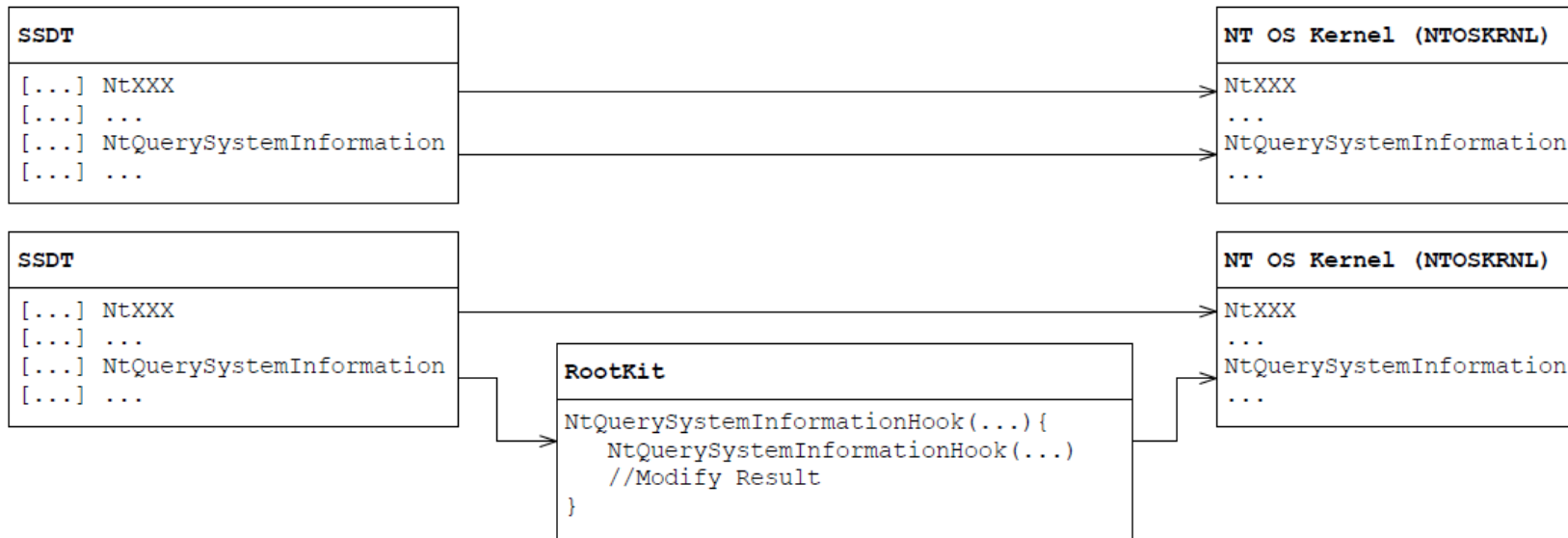
- Traditionally, rootkit functionality utilizes **operating system** interfaces to **conceal the presence** of malware (=stealth)
- Traditional rootkit variants (next slides): **User-mode** and **kernel-mode** rootkits
- Other rootkit variants:
 - **Bootkit** - runs in real mode (and kernel mode after booting)
 - Infects startup code (boot sector) to take control of the boot process
 - Can subvert the kernel and its security protections
 - **Hypervisor level** rootkits
 - Exploits **hardware virtualization** features such as Intel VT or AMD-V
 - Runs in **Ring -1** and hosts the target operating system **as a VM**
 - Does **not have to make any modifications to the kernel** of the target
 - **Firmware** and **hardware** rootkits
 - Run on routers, network card, hard drive or system BIOS

Rootkits – User Mode



- Run in **user space** and work best with **administrative privileges**
- Alter security settings and hide processes, files, system drivers, network ports, and even system services
- Hiding techniques:
E.g., **function hooking** of widely used API in every process

Rootkits – Kernel-Mode



- Loaded as **device drivers** / loadable kernel modules
=> requires to be able to install a driver (kernel module)
- **Many** different **hiding techniques**
- E.g., **SSDT** (System Service Descriptor Table) hooking

Rootkits – Kernel-Mode

- Protections:

- Windows Driver Signing – Starting with Windows 10, drivers must be submitted to, checked and signed by Microsoft
- PatchGuard (all x64 Win OS since XP) protects from SSDT-like rootkits by (trying to) preventing modifications of data structures like the SSDT
- Device Guard locks a device down so that it can only run trusted applications

- Device Guard:

- Available on Windows 10 Enterprise or higher
- Operating system only runs code that's signed by trusted signers
- Who's a trusted signer is defined by your Code Integrity policy through specific hardware and security configurations

Detecting Rootkits

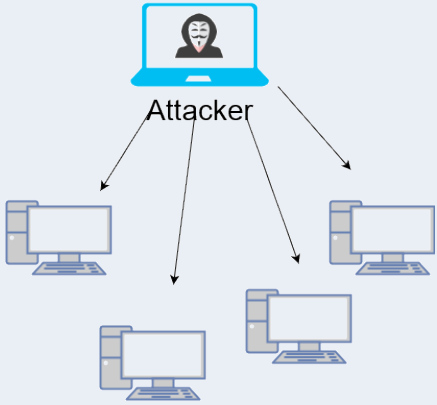
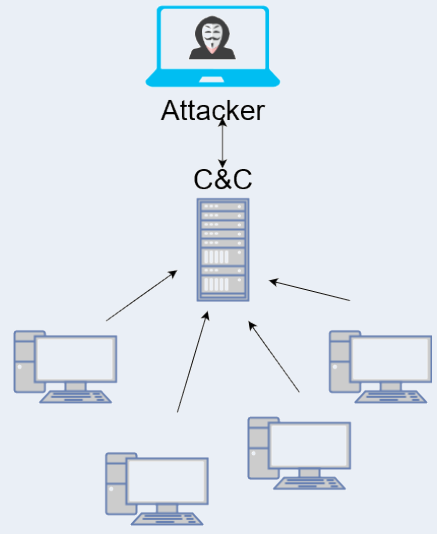
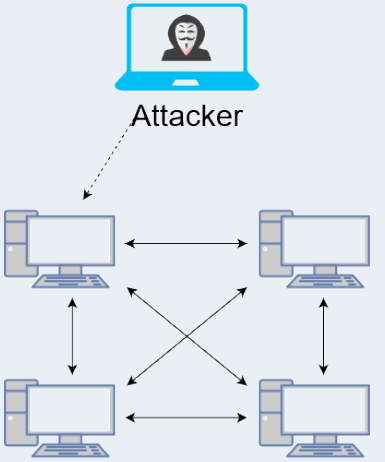
- Based on the «concept» used for hiding
 - E.g., look for alteration of the SSDT
- Detection of sophisticated rootkits at ring 0 or lower is difficult
- Timing analysis as a generic detection method for rootkits
 - Hooked/modified operations show a different timing behavior
 - Baseline to compare to is the challenge
 - In case of hypervisor level rootkits, external time sources must be used

Malware Communication

Introduction

- Many malware types **need to communicate** with the attacker(s) or other malware instances to fulfil their purpose
- Communication consists of the **communication architecture** and the **properties of the communication channel** itself
- Important goals for malware communication:
 - Communication should be **resilient** vs. **blocking/takedowns**
 - Communication should be **hard to detect** and/or **identify**

Communication Architectures

	Direct	Client-Server	P2P
Communication initiated by	attacker	malware	attacker/malware
Firewall compatibility	low	high	low-medium
Resilience / Ease of takeout	easy [hard]	easy [hard]	hard
Architecture Outline			

Client-Server – Mitigations of Single Point of Failure

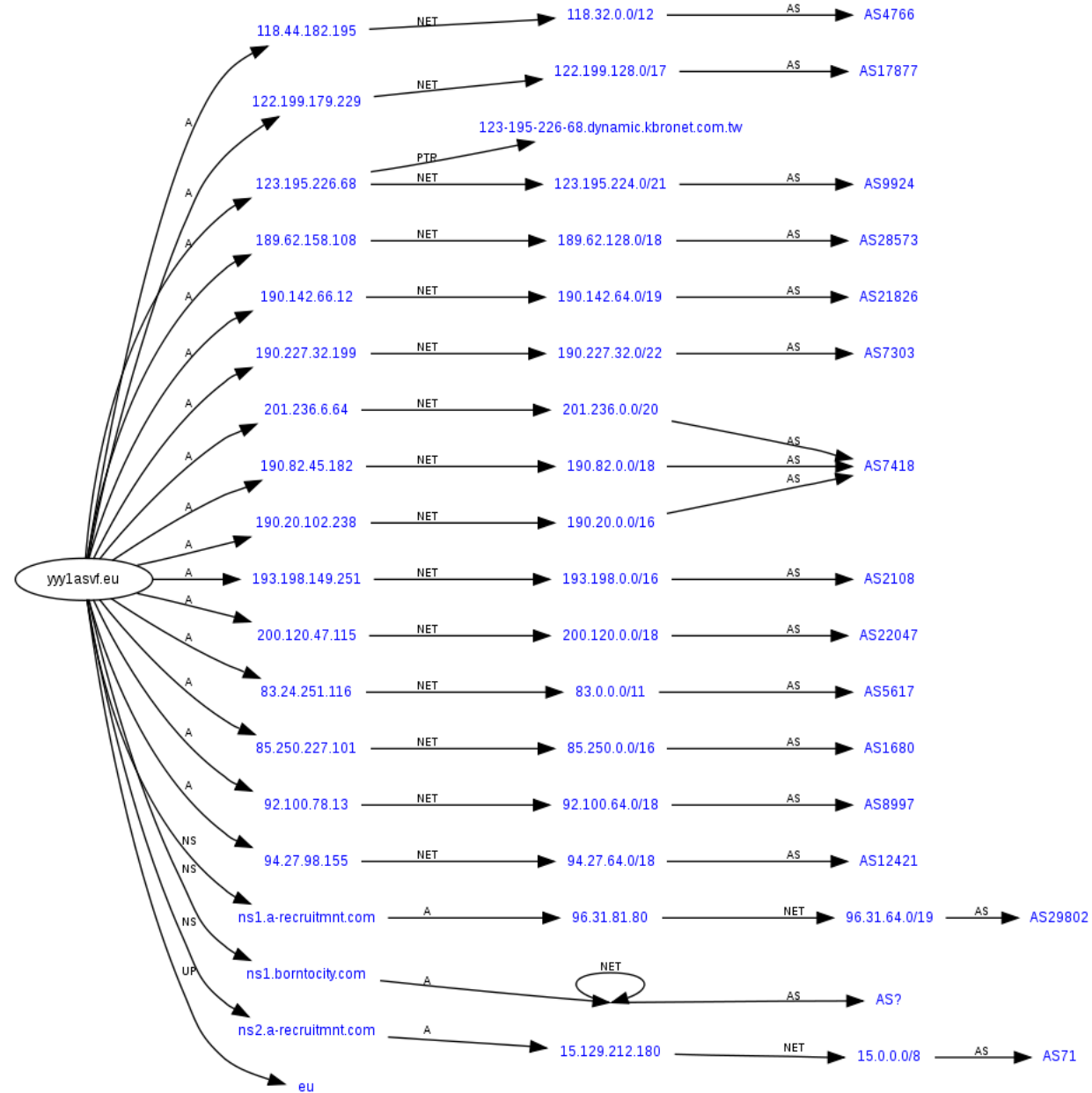
- If the **IP/domain/URL** of the C2 server is known, we can **take it down** (e.g., arrest the attacker), **block** connections to it or **sinkhole*** it
- **Some mitigations:**
 - Change the IP(s) on a regular basis => **Fast Flux** (e.g., Fluxer Botnet in 2015)
 - Change the domain name on a regular basis => **DGAs** (e.g., Emotet malware)
 - Malware connects to a seemingly clean domain => **Domain Fronting**
 - Communicate over channels like Dropbox or Evernote (e.g., BKDR_VERNOT.A in 2013)
 - IRC based botnets multiplexing multiple channels (e.g., DorkBot botnet in 2015)
- **Mitigations for the mitigations?.**
 - E.g., partnering with registrars where the cybercriminals bought the domain names associated with the botnet (it's C2 server(s))

Fast Flux – What it is

- Basic idea: A single fully qualified domain name, where IP addresses are swapped in and out with high frequency (TTL <300s), through changing DNS records.
- Fast flux is a DNS technique to hide a machine behind an ever-changing network of compromised hosts acting as proxies.
 - Makes malware networks more resistant to IP based blocking
 - The Storm Worm (2007) was one of the first malware variants to make use of this technique.
 - It can also refer to the combination of peer-to-peer networking, distributed command and control, web-based load balancing and proxy redirection
- How to defend?
 - Block the fully qualified domain or take it down with the help of the registrars/providers where the domains were registered

Fast Flux - Visualization

DNS [Robtex](#)
Analysis of a
Fast flux
domain



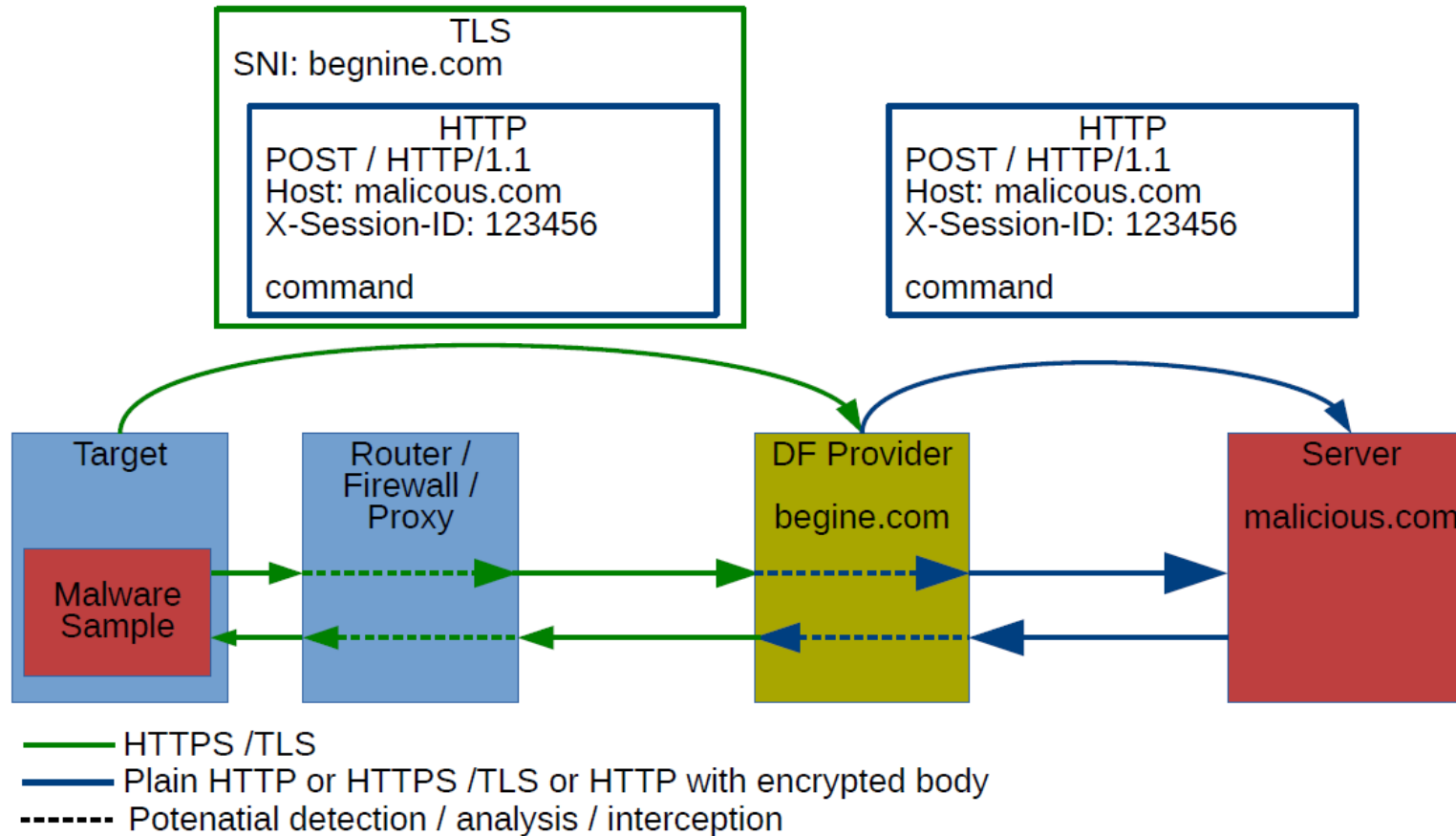
Domain Generation Algorithm (DGA)

- Thwart “bad domain” list defense with domain generation algorithm
- Used to periodically generate many domain names that can be used as rendezvous points with their C2 servers.
- The large number of potential rendezvous points makes it difficult for law enforcement to effectively shut down botnets

```
def generate_domain(year, month, day):  
    """Generates a domain name for the given date."""  
    domain = ""  
  
    for i in range(16):  
        year = ((year ^ 8 * year) >> 11) ^ ((year & 0xFFFFFFFF0) << 17)  
        month = ((month ^ 4 * month) >> 25) ^ 16 * (month & 0xFFFFFFFF8)  
        day = ((day ^ (day << 13)) >> 19) ^ ((day & 0xFFFFFFF0) << 12)  
        domain += chr(((year ^ month ^ day) % 25) + 97)  
  
    return domain
```

Domain Fronting

- Communicate with your C2 server using a totally legitimate domain

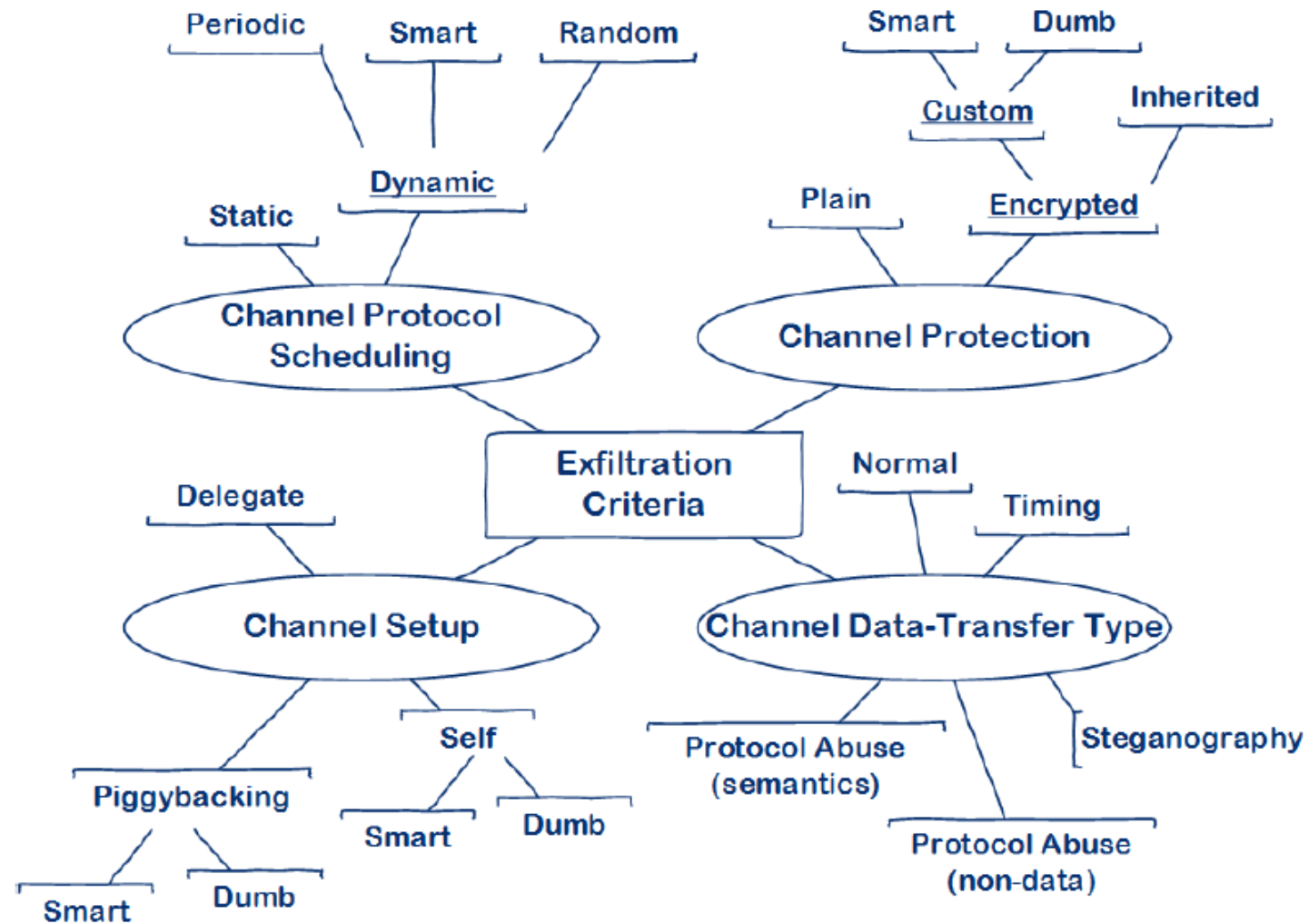


Malware Communication

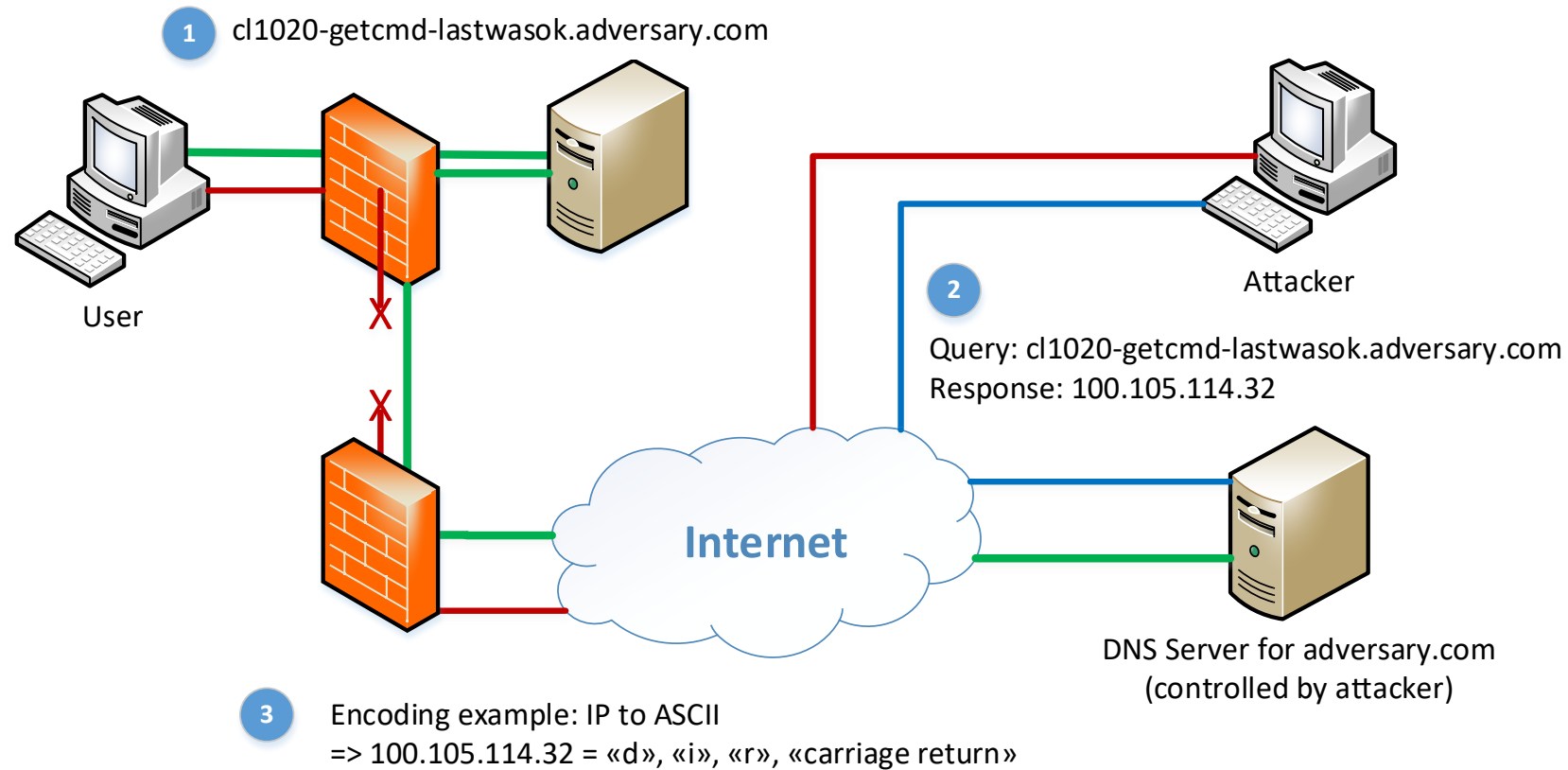
- Most common: **HTTPS** with or without **content-level encryption**
- Advanced concept: Smart communication/exfiltration
 - **What protocols** are used [HTTP, HTTPS, SSH, ...]?
 - **What applications** are used [Dropbox, Google docs, Citrix, ...]?
 - **How much data** is sent/received **and when**?

=> **Communication should adapt to what is "normal"**
- **Side Channels** to send out data even if communication is over an encrypted tunnel only (e.g., VPN only)
 - E.g., when machine is on WLAN, malware could delay packets to create specific packet inter-arrival patterns
- **DNS Covert Channel** when isolated from the Internet might work

Attributes of Communication Channels



Communication – Covert Channel using DNS

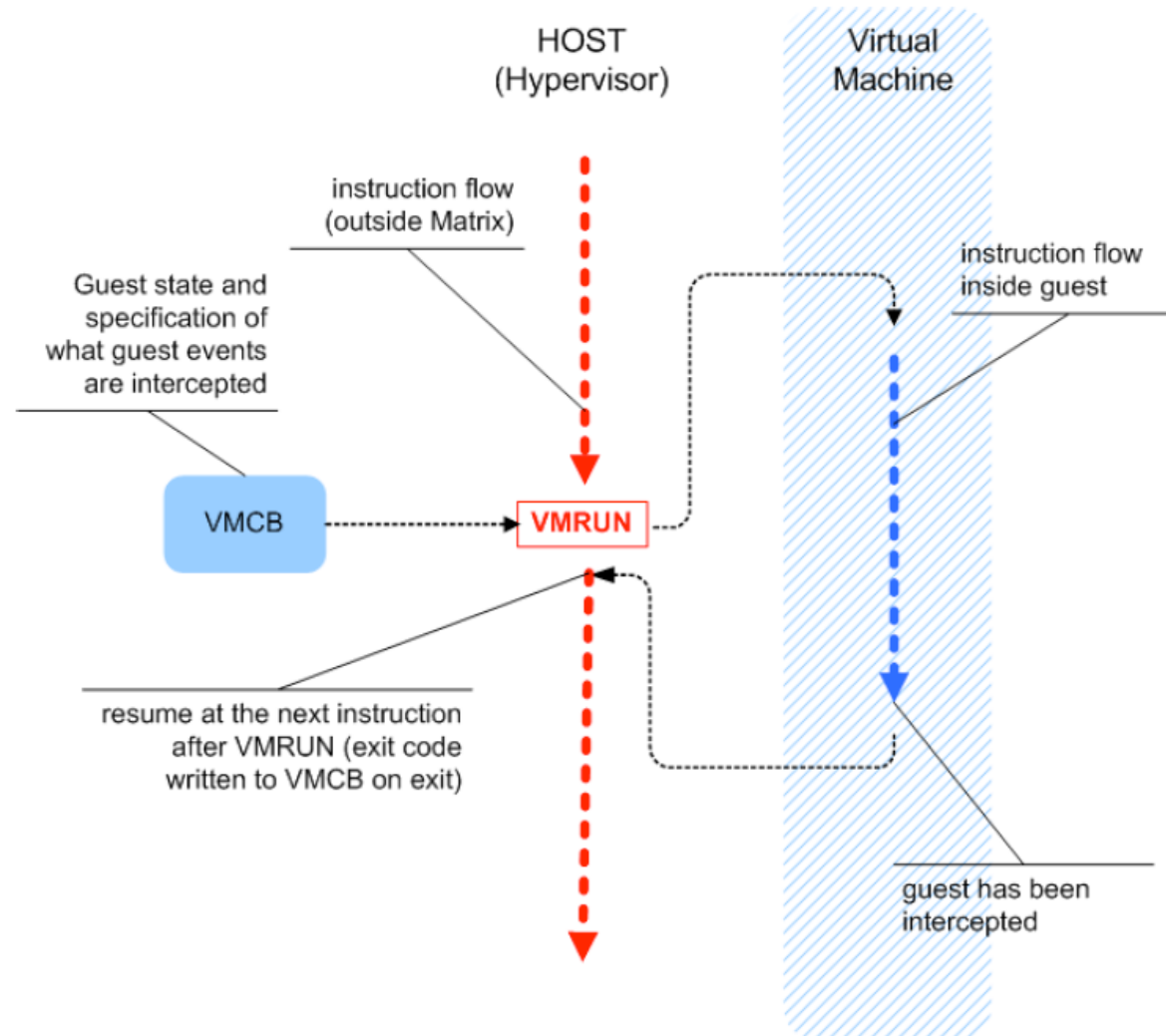


Appendix

Famous Example of a Hypervisor-level Rootkit

- Blue Pill rootkit for Windows Vista x64
- Introduced by researcher Joanna Rutkowska in 2006
- Blue Pill idea:
 - Exploit AMD64 SVM extension (later also Intel VT) to move the operating system into a virtual machine 'on-the-fly'
 - Provide thin hypervisor to control the OS
 - Hypervisor is responsible for controlling interesting events inside guest OS
- The VMRUN instruction is the heart of AMD64 SVM (and Blue Pill)

Blue Pill – The VMRUN Instruction (heart of AMD64 SVM)



Blue Pill – The Basic Idea (simplified)

